

SHUNYA OS — Backend Architectural Duplication Report (LX-06 Audit)

Date: 2026-08-05
Scope: /home/shunya-deploy/shunya_os/app/
Files scanned: 206 Python files across all directories
Classification: Canonical / Duplicate / Legacy / Experimental / Dead

1. AUTHENTICATION — ⚠ MASSIVE DUPLICATION (4+ parallel systems)

Component	Classification	Notes
app/auth.py — TeamMember model, AuthLayer, SHA256 hashing	Legacy	Table <code>team_members</code> . Still used by <code>auth_routes</code> for session login. Will be retained until all users migrate to kernel identity.
app/auth_routes.py — <code>auth_bp</code> , <code>login_required</code> , <code>/login</code> , <code>/logout</code> , <code>/team/</code> , <code>/api/v1/auth/signup</code>	Legacy	Actively wired. <code>login_required</code> decorator used dozens of places. Email verification endpoints DUPLICATED with <code>production/auth/</code> .
app/auth_oauth.py — <code>oauth_bp</code> , Google+GitHub OAuth	Duplicate (Migration bridge)	Creates identities via <code>IdentityRepository</code> . Sets both <code>session['user_id']</code> AND <code>session['identity_id']</code> . Bridges legacy→kernel.
app/production/auth/password_reset_routes.py	Duplicate	Registers on <code>auth_bp</code> . In-memory token store. Duplicates pattern already in <code>auth_routes.py</code> .
app/production/auth/email_verification_routes.py	Duplicate	Registers <code>/request-verification</code> on <code>auth_bp</code> . <code>auth_routes.py</code> already has <code>/api/v1/auth/request-verification</code> . Two parallel in-memory token stores.
app/production/auth/mfa_routes.py	Duplicate	In-memory MFA state. Same pattern.
app/production/auth/session_routes.py	Duplicate	Session revocation, in-memory version tracking.
app/authz/ — Role, OrgMemberRole, permission system	Canonical	Newer, cleaner authorization engine. Separates auth (who you are) from authorization (what you can do). Coexists with legacy <code>UserRole</code> .
app/security/jwt.py	Canonical	JWT utilities.

Key overlap: Two email verification implementations (`auth_routes.py` lines 270-305 AND `production/auth/email_verification_routes.py`), both on the same `auth_bp` blueprint. Different in-memory stores.

2. IDENTITY — ⚠ TRIPLICATION

Component	Classification	Notes
app/kernel/identity.py — <code>SHUNYAIdentity</code> , <code>AuthMethodType</code> , <code>IdentityStore</code>	Canonical	Kernel-level identity contract. Frozen by design.
app/production/identity_repository.py — <code>IdentityRepository</code> , <code>SHUNYAIdentityModel</code>	Canonical (Bridge)	Persistence bridge. Exposes both legacy <code>find_by_auth()</code> and new <code>find_by_auth_core()</code> APIs.
app/production/identity/ — <code>org_routes</code> , <code>user_routes</code> , <code>workspace_routes</code> , <code>invitation_routes</code> , <code>lifecycle_routes</code> , <code>onboarding_routes</code> , <code>switch_routes</code>	Canonical	Milestone X identity CRUD. All registered on <code>identity_bp</code> at <code>/orgs</code> .
app/shunya/identity/engine.py — <code>IdentityEngine</code>	Duplicate	Implements resolver, lifecycle, normalizer. Overlaps heavily with <code>kernel.identity</code> + <code>identity_repository</code> .
app/shunya/identity/models.py — <code>IdentityType</code> , <code>IdentityStatus</code> , <code>ResolutionStatus</code> enums	Duplicate	Different enum values from <code>kernel/identity.py</code> (<code>AuthMethodType</code>). Parallel enum hierarchy.
app/gkf/identity.py — GKF identity (collection/volume/chapter)	Canonical (for GKF)	Document-structural identity. Different domain, but same concept "identity" used differently.
app/auth.py <code>TeamMember</code>	Legacy	The original "identity as email+password".

3. WORKSPACE MODELS — ⚠ TRIPLICATION

Component	Classification	Notes
app/objects/models.py — Workspace (table <code>sh_workspaces</code> , <code>id='spc_xxx'</code>)	Canonical	Phase 0 foundation. Used by object routes.
app/production/identity/workspace_model.py — Workspace (table <code>workspaces</code> , <code>id</code> Integer, FK to <code>tenants</code>)	Duplicate	Milestone X production workspace. Different table, different schema, same concept.
app/workspace/models.py — Experience catalog (no SQL model)	Canonical	Experience framework only, not a persistence model.

4. ENGINES — SIGNIFICANT OVERLAP (12+ engine classes)

Component	Classification	Notes
<code>app/reality_engine/engine.py</code> — RealityEngine (LX-02)	Canonical	Highest-level composition engine. Composes events, attention, objects, execution, awareness, graph. "Single source from which interface derives."
<code>app/intelligence/</code> — Explainable Intelligence (observation, reasoning, insight, confidence, inspector, scenario)	Canonical	Explainability pipeline with provenance. Separate domain.
<code>app/awareness/engine.py</code> — Operational Awareness (8 sub-engines)	Experimental	Reads from execution, execution_intelligence.
<code>app/cognitive/engine.py</code> — Cognitive Validation (reasoning graphs, contradictions, replay)	Experimental	Validates cognitive pipeline. Heavy overlap with intelligence/ in concepts.
<code>app/execution_intelligence/engine.py</code> — Execution Intelligence (7 engines: health, risk, timeline, dependency, next_action, portfolio)	Experimental	Overlaps with awareness/ and executive/ in health/risk concepts.
<code>app/executive/engine.py</code> — Executive Intelligence	Experimental	Synthesizes organizational intelligence. Depends on orchestration, decision, cognitive.
<code>app/orchestrator/engine.py</code> — OrchestratorEngine	Experimental	Pipeline orchestration.
<code>app/prediction/engine.py</code> — Prediction & Simulation	Experimental	Read-only predictions.
<code>app/collaboration/engine.py</code> — Collaboration (presence, sessions, shared)	Experimental	Multi-user runtime.
<code>app/organizational/engine.py</code> — Organizational Intelligence	Experimental	
<code>app/learning_intelligence/engine.py</code> — Learning Intelligence	Experimental	
<code>app/orchestration/</code> — cycle, queue, signal, sync	Experimental	Orchestration runtime layer.

Overlap pattern: intelligence/, awareness/, cognitive/, execution_intelligence/, executive/ all define engine classes with overlapping concepts (health scoring, risk detection, insight generation, attention scoring). Each has its own model classes and its own "engine" with `get_*`() singleton accessors.

5. RUNTIMES — SIGNIFICANT OVERLAP

Component	Classification	Notes
<code>app/outcome_engine.py</code> — OutcomeEngine, WorkflowEngine	Canonical	Executes named outcomes via intent/name. Used by <code>/api/outcomes/execute</code> .
<code>app/execution/runtime.py</code> — OutcomeRuntime	Duplicate	Also manages outcomes from acceptance→completion. Different code path. Overlaps with outcome_engine.py.
<code>app/workspace_runtime.py</code> — WorkspaceRuntime	Canonical	Object registry, runtime-driven workspace state.
<code>app/decision_runtime/</code> — Decision middleware	Experimental	Decisions, policies, commitments.
<code>app/planning/runtime.py</code>	Experimental	Planning runtime.
<code>app/temporal/runtime.py</code>	Experimental	Temporal runtime.
<code>app/orchestration/runtime.py</code>	Experimental	Orchestration runtime.
<code>app/graph_universal/runtime.py</code>	Experimental	Universal graph runtime.
<code>app/space/runtime.py</code>	Experimental	Universal space runtime.
<code>app/organization/runtime.py</code>	Experimental	Organization runtime.
<code>app/cortex/runtime.py</code> — Cortex middleware	Experimental	Organizational cortex.

Key overlap: outcome_engine.py and execution/runtime.py both implement outcome execution. The outcome_engine is invoked from routes.py (`/api/outcomes/execute`), while execution/runtime.py OutcomeRuntime is a separate implementation.

6. AI / LLM / INFERENCE — TRIPLICATION

Component	Classification	Notes

<code>app/ai/</code> — ai_bp, chat endpoint, provider registry, copilot, context, prompts	Canonical	Active AI chat API. Provider chain: Groq→Gemini→OpenRouter→Cloudflare→HF→Local.
<code>app/llm/</code> — LLMRuntimeService	Duplicate	Separate provider adapter pattern. Has ModelRun persistence model. Does not use <code>app/ai/provider.py</code> .
<code>app/inference/</code> — InferenceControlPolicy	Experimental	Control-plane only (policy decisions). No provider calls.

Key overlap: `app/ai/provider.py` has an LLMProvider abstraction with 6+ backends. `app/llm/_init_.py` has its own LLMRuntimeService with its own FakeProviderAdapter. Two separate LLM abstraction layers.

7. SEARCH — ⚠️ DUPLICATION

Component	Classification	Notes
<code>app/search.py</code> — UniversalSearch class	Legacy	SQL LIKE queries across leads, payments, invoices, suppliers, knowledge, media.
<code>app/search/</code> — search_bp, providers, context builder	Canonical	DuckDuckGo web search, clean provider abstraction, company context builder.

8. NOTIFICATIONS — ⚠️ DUPLICATION

Component	Classification	Notes
<code>app/notifications.py</code> — NotificationManager	Legacy	Works with <code>app.models.Notification</code> (old model).
<code>app/integration/models.py</code> — Notification model (table <code>m6_notifications</code>)	Canonical	Newer model with identity_id, notification_type, channels, email dispatch status.

9. BLUEPRINT EXPLOSION — 34 registered blueprints

All blueprints registered in `create_app()` (most in `_init_.py`):

Blueprint	URL Prefix	Classification
<code>auth_bp</code>	(none)	Legacy
<code>main</code>	(none)	Legacy
<code>api</code>	(none)	Legacy
<code>objects_bp</code>	<code>/api/v1/objects</code>	Canonical
<code>objects_upload_bp</code>	(implicit)	Canonical
<code>production_bp</code>	<code>/api/v1</code>	Canonical
<code>shunya_bp</code>	(none)	Canonical
<code>workspace_bp</code> (from <code>workspace_routes.py</code>)	<code>/workspace</code>	Dead (just serves SPA shell)
<code>founder_bp</code>	(implicit)	Canonical
<code>upload_bp</code>	(implicit)	Canonical
<code>search_bp</code>	<code>/api/v1</code>	Canonical
<code>jobs_bp</code>	(implicit)	Canonical
<code>intention_bp</code>	(implicit)	Canonical
<code>file_bp</code>	(implicit)	Canonical
<code>for1_bp</code>	<code>/for1</code>	Legacy
<code>for2_bp</code>	<code>/for2</code>	Legacy
<code>reality_bp</code>	(implicit)	Canonical
<code>relationship_bp</code>	(implicit)	Canonical
<code>authz_bp</code>	(implicit)	Canonical
<code>finance_bp</code>	(implicit)	Canonical
<code>onboarding_bp</code>	(implicit)	Canonical
<code>oauth_bp</code>	<code>/api/v1/auth</code>	Duplicate
<code>integration_bp</code>	(implicit)	Canonical

cloudinary_bp	(implicit)	Canonical
pdf_bp	(implicit)	Canonical
razorpay_bp	(implicit)	Canonical
execution_bp	(implicit)	Experimental
automation_bp	(implicit)	Canonical
events_bp	/api/v1	Canonical
intelligence_bp	(implicit)	Canonical
communication_bp	(implicit)	Canonical
enterprise_bp	(implicit)	Canonical
ai_bp	/api/v1/ai	Canonical
genesis_bp	(implicit)	Canonical
space_bp	/api/v1/space	Experimental
workspace_bp (from app.workspace)	/api/v1/workspace	Canonical

10. SUMMARY OF OVERLAP CLUSTERS

Highest priority for consolidation:

Cluster	Files	Action
Auth (4 systems)	auth.py , auth_routes.py , auth_oauth.py , production/auth/* , authz/	Merge into authz/ as canonical. Deprecate legacy TeamMember in favor of kernel Identity.
Identity (3 systems)	kernel/identity.py , production/identity_repository.py , shunya/identity/	shunya/identity/ is fully duplicated. Remove it.
Workspace (3 models)	objects/models.py , production/identity/workspace_model.py	Unify into one model. Production model is newer; Phase 0 model has spc_xxx IDs.
Engines (12+ engines)	intelligence/ , awareness/ , cognitive/ , execution_intelligence/ , executive/ , orchestrator/	Consolidate into RealityEngine as canonical. All others are experimental spin-offs.
Outcome (2 impls)	outcome_engine.py , execution/runtime.py	OutcomeEngine is canonical; OutcomeRuntime is overlapping.
AI/LLM (2 abstractions)	ai/provider.py , llm/__init__.py	LLMRuntimeService duplicates provider pattern from ai/. Merge or remove.
Search (2 impls)	search.py , search/	Remove legacy search.py (UniversalSearch).
Notifications (2 models)	notifications.py , integration/models.py	Retain integration/models.py . Remove notifications.py NotificationManager.
Email verification (2 routes)	auth_routes.py , production/auth/email_verification_routes.py	Two routes on same blueprint. Kill the production/auth version.
Middleware (3 auth checks)	__init__.py lines 378-388, 672-713; auth_routes.py login_required	Three separate auth middleware layers on the same app. Unify into one.